

CanlıFal — Gerçek Zamanlı (Realtime) Sistemler Teknik Dokümantasyonu

Amaç: Flutter mobil uygulaması ile web backend'inin **birebir aynı** çalışması için gereken tüm gerçek zamanlı sistemlerin (SSE, event, payload, heartbeat, reconnect, presence, chat, voice room, live stream, TRTC token, join/leave, PK, co-host, music queue, gift) eksiksiz teknik referansı.

Kaynak: Bu doküman tahmine değil, **gerçek kaynak koda** dayanır. Her event için gerçek payload alanları ve örnek JSON verilmiştir. Kodda bulunmayan hiçbir alan uydurulmamıştır; bir davranış eksik/yarım ise açıkça belirtilmiştir.

Base URL: <https://canlifal.com>

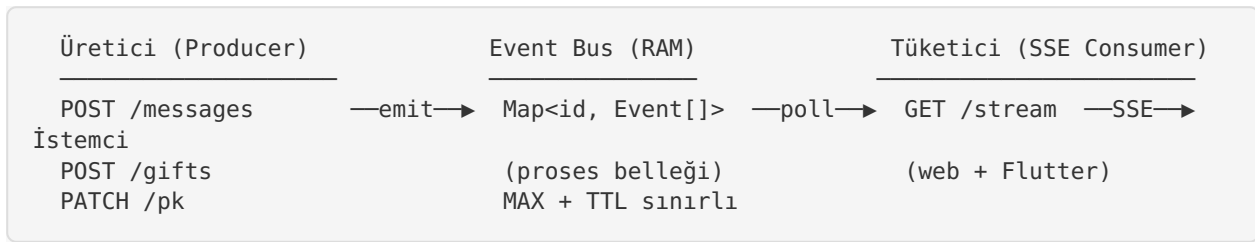
İçindekiler

- [Genel Mimari](#)
- [SSE Endpoint'leri](#)
- [Event İsimleri — Ana Tablo](#)
- [Heartbeat](#)
- [Reconnect Mantığı](#)
- [Presence Sistemi](#)
- [Online Kullanıcı Sistemi](#)
- [Chat Sistemi](#)
- [Voice Room Akışı](#)
- [Live Stream Akışı](#)
- [Tencent RTC \(TRTC\) Token Üretimi](#)
- [Join / Leave Akışları](#)
- [PK Battle Sistemi](#)
- [Co-Host \(Ortak Yayın\) Sistemi](#)
- [Music Queue \(DJ\) Sistemi](#)
- [Gift Event Sistemi](#)
- [Canlı Fal Seansı \(Session Room\) Akışı](#)
- [WebRTC Signaling](#)
- [Flutter ↔ Web Parite Kontrol Listesi](#)

1. Genel Mimari

1.1 SSE + In-Memory Event Bus

Gerçek zamanlı iletişim **Server-Sent Events (SSE)** ile yapılır. WebSocket kullanılmaz. Mimari şu şekildedir:



- **Üretici uçlar** (POST/PATCH) veritabanı işlemini yaptıktan sonra `emit*Event(id, type, data)` çağırır. Event, RAM'deki bir Map tamponuna (buffer) yazılır.
- **SSE uçları** (GET .../stream) bu tamponu belirli aralıklarla (poll) tarar (`get*EventsSince(id, timestamp)`) ve yeni olayları istemciye `data: {...}\n\n` biçiminde iletir.
- Bu “polling destekli SSE” yaklaşımıdır: iletim gecikmesi tampon poll aralığı kadardır (genelde 1-2 sn). **Gerçek push değildir.**

1.2 Event Bus Tamponları (buffer) — Kod Gerçekleri

Event Bus (lib)	Anahtar	Maks. olay	TTL	Cleanup	Kullanım
chat-events.ts	roomId	200	2 dk	30 sn	Chat, voice room, gift, pk, system, typing
stream-events.ts	streamId	100	(poll ile)	—	Live stream olayları
room-events.ts (session)	sessionId	100	5 dk	60 sn	Canlı fal seansı olayları
room-events.ts (teller)	tellerId	100	5 dk	60 sn	Falciya gelen seans talepleri
chat-dj-events.ts	roomId	1 (son durum)	5 dk	5 dk	Müzik/DJ durumu

Önemli: Event bus proses belleğindedir. Sunucu yeniden başlarsa tamponlar sıfırlanır; bu nedenle her SSE bağlantısı açılışta **initial state** (mevcut durum) gönderir ve istemci gerekirse REST ile tam durumu (/state , /join-room) tazeler.

1.3 Dual-Auth (İkili Kimlik Doğrulama)

Tüm realtime uçlar hem **mobil (Flutter)** hem **web** istemciyi destekler:

```
const mobileUser = await authenticateRequest(request) // JWT Bearer (Flutter)
const session = !mobileUser ? await getServerSession(authOptions) : null // NextAuth (web)
const currentUserId = mobileUser?.id || session?.user?.id
```

- **Flutter:** Authorization: Bearer <accessToken> başlığı gönderir. Access token 7 gün, refresh token 30 gün geçerlidir.
- **Web:** NextAuth session cookie kullanır.
- Bazı uçlar (ör. video stream izleme) misafir (viewer_xxx / guest_xxx) kullanıcıya da izin verir.

1.4 Ortak SSE Yanıt Başlıkları

Tüm */stream uçları şu başlıklarla yanıt verir:

```
Content-Type: text/event-stream
Cache-Control: no-cache, no-transform
Connection: keep-alive
X-Accel-Buffering: no
```

1.5 SSE Mesaj Formatı

Her olay tek satırlık bir data: bloğudur ve \n\n ile biter:

```
data: {"type":"connected","roomId":"abc123"}

data: {"type":"messages","messages":[ ... ]}

: heartbeat
```

- data: ile başlayan satır → JSON yük (payload).
- : ile başlayan satır → **yorum/heartbeat** (JSON değildir, istemci yok saymalı, yalnızca bağlantı canlı tutmak içindir).

2. SSE Endpoint'leri

Toplam **5 adet** gerçek zamanlı SSE endpoint'i vardır. Hepsi GET metodu, dual-auth, runtime = 'nodejs', dynamic = 'force-dynamic'.

#	Endpoint	Amaç	Poll aralığı	Yayınlanan olaylar
1	GET /api/chat/rooms/{roomId}/stream	Chat + Voice Room (ana kanal)	2 sn	connected , messages , system , gift , pk , room_event , presence , dj , typing
2	GET /api/video-streams/{streamId}/stream	Canlı video yayını	1 sn	connected , viewerCount , streamMessage , gift , streamEnded
3	GET /api/room/{sessionId}/stream	Canlı fal seansı odası	1 sn	connected + ham seans olayları (message , timer_started , time_extended , session_ended)
4	GET /api/fortune-tellers/sessions/stream	Falciya gelen seans talepleri	3 sn	connected , pending_sessions , session_request , session_cancelled
5	GET /api/notifications/stream	Kullanıcı bildirimleri	5 sn	connected , notification

Not: chat/rooms/{roomId}/stream en zengin kanaldır. Voice room, chat, gift, PK, music (DJ) ve moderasyon olaylarının **tamamı** bu tek kanaldan akar. Flutter için voice room ve chat ekranı **aynı** SSE bağlantısını paylaşır.

2.1 Bağlantı açılışında gönderilen ilk olay (connected)

Her endpoint bağlantı açılır açılmaz bir connected olayı gönderir. İçeriği endpoint'e göre değişir:

Chat/Voice (/chat/rooms/{roomId}/stream):

```
{ "type": "connected", "roomId": "clr9x2abc" }
```

Video Stream (/video-streams/{streamId}/stream):

```
{ "type": "connected", "streamId": "vs_8823" }
```

Hemen ardından ilk izleyici sayısı gönderilir:

```
{ "type": "viewerCount", "streamId": "vs_8823", "viewerCount": 42 }
```

Session Room (/room/{sessionId}/stream):

```
{  
  "type": "connected",  
  "sessionId": "ls_5521",  
  "isUser": true,  
  "isTeller": false,  
  "status": "active",  
  "timerStarted": true,  
  "timerStartedAt": "2026-07-23T10:15:00.000Z",  
  "maxMinutes": 15,  
  "minutesUsed": 3  
}
```

Teller Sessions (/fortune-tellers/sessions/stream):

```
{ "type": "connected", "tellerId": "ft_331", "isOnline": true }
```

Notifications (/notifications/stream):

```
{ "type": "connected", "unreadCount": 5 }
```

3. Event İsimleri — Ana Tablo

Aşağıda her SSE kanalındaki tüm olay adları ve tetikleyen (producer) uçlar listelenmiştir.

3.1 Chat/Voice kanalı (/chat/rooms/{roomId}/stream)

SSE type	İç event bus tipi	Tetikleyen uç	Açıklama
connected	—	(bağlantı açılışı)	İlk el sıkışma
messages	message	POST /chat/rooms/{roomId}/messages , POST /live/message	Yeni sohbet mesaj(lar)ı (dizi)
system	system	POST /chat/rooms/{roomId}/moderation	Moderasyon (mute/ban/kick/clear/announcement)
gift	gift	POST /chat/rooms/{roomId}/gifts , POST /live/gift/send	Hediye gönderimi
pk	pk	POST /chat/rooms/{roomId}/pk , .../pk/score , POST /live/pk	PK Battle olayları
room_event	room	presence/seats/voice/transfer-ownership uçları	Voice room olayları (join/left/mic/seat/owner/closed)
presence	(DB)	(SSE 10 sn'de bir DB'den)	Aktif kullanıcı listesi
dj	(dj store)	music/song-request/dj uçları	Müzik kuyruğu durumu
typing	typing	POST /chat/rooms/{roomId}/typing	Yazıyor göstergesi

3.2 Video Stream kanalı (/video-streams/{streamId}/stream)

SSE type	Tetikleyen uç	Açıklama
connected	(bağlantı açılışı)	İlk el sıkışma
viewerCount	join , leave , DELETE	Anlık izleyici sayısı
streamMessage	POST .../comments , POST .../messages	Yayın sohbet mesajı
gift	POST .../gifts , POST /live/gift/send	Hediye
streamEnded	PATCH .../ (status=ended), POST .../end	Yayın bitti

3.3 Session Room kanalı (/room/{sessionId}/stream)

İç event tipi	Tetikleyen uç	Açıklama
message	POST /room/{sessionId}/messages	Seans içi metin mesajı
timer_started	PATCH /room/{sessionId} (start_timer)	Süre sayacı başladı
time_extended	PATCH /room/{sessionId} (teller_add_time / add_time)	Süre uzatıldı
session_ended	PATCH /room/{sessionId} (end)	Seans bitti

KRİTİK NOT (Flutter parity): Session Room SSE kanalı, olayı `event.data` olarak **ham** iletir — yani gönderilen JSON'da ayırt edici bir `type / event` alanı **yoktur** (yalnızca `connected` olayında `type` vardır). İstemci, olay tipini **mevcut alanlara bakarak** ayırt etmelidir (örn. `timerStartedAt` varsa `timer_started` , `actualMinutesUsed` varsa `session_ended`). Bu, kodun mevcut gerçek davranışdır.

3.4 Teller Sessions kanalı (/fortune-tellers/sessions/stream)

SSE type	Tetikleyen uç	Açıklama
connected	(bağlantı açılışı)	İlk el sıkışma
pending_sessions	(açılış + 15 sn DB fallback)	Bekleyen talep listesi
(ham) session_request payload	POST /fortune-tellers/session	Yeni seans talebi
(ham) session_cancelled payload	PATCH /fortune-tellers/sessions/{sessionId}	Talep iptal/red

3.5 Notifications kanalı (/notifications/stream)

SSE type	Açıklama
connected	İlk el sıkışma + okunmamış sayısı
notification	Yeni bildirim (5 sn'de bir DB kontrolü)

4. Heartbeat

İki farklı "heartbeat" kavramı vardır; karıştırılmamalıdır:

4.1 SSE Heartbeat (sunucu → istemci)

Tüm SSE endpoint'leri, ara katman/proxy bağlantıyı kapatmasını diye **15 saniyede bir** yorum satırı gönderir:

```
: heartbeat
```

- Bu bir JSON **değildir**. İstemci `data:` ile başlamayan satırları yok saymalıdır.
- EventSource kullanan istemciler için bu otomatik olarak yok sayılır.
- Flutter'da ham SSE parse ediliyorsa: `if (line.startsWith(':')) continue;`

4.2 İstemci Heartbeat (istemci → sunucu) — Presence/Liveness

Flutter istemcisi, oda/yayında **canlı** görünmek için **10 saniyede bir** POST atar:

İstek:

```
POST /api/live/heartbeat
Authorization: Bearer <jwt>
Content-Type: application/json
```

```
{ "roomId": "clr9x2abc", "roomType": "voice" }
```

roomType : "voice" (chat/voice room) veya "stream" (canlı yayın).

Yanıt:

```
{
  "success": true,
  "data": {
    "onlineCount": 37,
    "staleRemoved": 2,
    "serverTime": "2026-07-23T10:20:05.123Z"
  }
}
```

Sunucu tarafı davranış (kod gerçeği):

- voice → ChatPresence.lastSeen güncellenir; aktif VoiceSession.lastPing tazelenir.
- stream → VideoStreamViewer.joinedAt güncellenir (gerekirse leftAt=null ile yeniden katılım).
- **Otomatik temizlik:** 60 saniyeden fazla heartbeat atmayan katılımcılar otomatik "left" işaretlenir (staleRemoved bunların sayısıdır).
- onlineCount hesabı: voice için 5 dakikalık pencere (lastSeen >= now - 300s), stream için leftAt = null sayısı.

Özet zaman sabitleri: SSE keep-alive = **15 sn**, istemci liveness heartbeat = **10 sn**, stale temizleme eşiği = **60 sn**, presence aktiflik penceresi = **5 dk (300 sn)**.

5. Reconnect Mantığı

SSE bağlantısı koptuğunda istemci yeniden bağlanır. Sistem **timestamp tabanlı** çalışır; klasik SSE Last-Event-ID başlığı **kullanılmaz**.

5.1 Genel Reconnect Akışı

1. Bağlantı kopunca istemci GET .../stream 'e **yeniden bağlanır**.
2. Sunucu, yeni bağlantıda lastEventCheck = Date.now() ile başlar — yani **bağlantı anından önceki** olaylar tekrar gönderilmez.
3. Bu boşluğu kapatmak için sunucu **açılıştaki mevcut durumu (initial state)** gönderir:
 - Chat/Voice: ilk poll'de tam DJ durumu + presence + connected .
 - Video: connected + anlık viewerCount .
 - Session: connected içinde sayaç/durum bilgisi.
 - Teller: connected + pending_sessions listesi.
4. Kritik durum (mesaj geçmişi, katılımcılar, koltuklar) için istemci reconnect sonrası **REST snapshot** çeker:
 - Voice/Chat: GET /api/chat/rooms/{roomId}/state veya POST /api/live/join-room
 - Video: POST /api/live/join-room (roomType=stream)
 - Mesaj geçmişi: GET /api/chat/rooms/{roomId}/messages

5.2 Önerilen Flutter Reconnect Stratejisi

- **Exponential backoff:** 1s → 2s → 4s → 8s (maks. ~15s), her başarılı olayda sıfırla.
- SSE **heartbeat** (15 sn) alınmıyorsa bağlantı ölü kabul edilip yeniden kurulur (örn. 20 sn timeout).
- Reconnect sonrası **daima** REST snapshot çekilerek kaçırılan durum tazelenir (idempotent).

5.3 WebRTC/TRTC Reconnect (Signaling temizliği)

Sesli/görüntülü katmanda yeniden bağlanırken eski (stale) sinyaller temizlenir:

```
DELETE /api/room/signal?sessionId={sessionId} // o kullanıcının tüm sinyallerini
siler
```

- `video-streams/signal` tarafında sinyaller zaten 60 sn sonra otomatik geçersiz sayılır ve `DELETE` ile eski kayıtlar silinir.

6. Presence Sistemi

Presence (bulunma), bir kullanıcının bir voice/chat odasında **aktif** olduğunu izler. Kaynak tablo: `ChatPresence` (`roomId` , `userId` , `nickname` , `seatIndex` , `lastSeen`).

6.1 Presence Kaydı Oluşturma / Güncelleme (Join)

```
POST /api/chat/rooms/{roomId}/presence
Authorization: Bearer <jwt>
```

```
{ "nickname": "Yıldız", "seatIndex": -1 }
```

- `seatIndex` = -1 → koltukta değil (dinleyici). 0..14 → koltuk indeksi.
- Yeni katılımda (`isNewJoin`) sunucu **hayalet önleme (ghost prevention)** uygular: kullanıcı başka bir odada presence taşıyorsa oradan otomatik çıkarılır (tek oda kuralı) ve `user_joined` olayı SSE'ye yayılır.

SSE'ye yayılan olay (`room_event` → `user_joined`):

```
{
  "type": "room_event",
  "event": "user_joined",
  "roomId": "clr9x2abc",
  "userId": "u_7781",
  "name": "Yıldız",
  "image": "https://.../avatar.jpg",
  "ts": 1753267205123
}
```

6.2 Presence Listesi (SSE `presence` olayı)

Chat/Voice SSE kanalı **10 saniyede bir** DB'den aktif kullanıcıları çekip yayınlar:

```
{
  "type": "presence",
  "users": [
    {
      "id": "u_7781",
      "name": "Yıldız",
      "image": "https://.../avatar.jpg",
      "nickname": "Yıldız",
      "lastSeen": "2026-07-23T10:20:00.000Z",
      "seatIndex": 2,
      "micOn": true,
      "chatRole": "op",
      "roleSymbol": "🛡️",
      "roleLevel": 3,
      "isAdmin": false
    }
  ]
}
```

- micOn : aktif VoiceSession (isActive=true) varsa true .
- chatRole : superadmin | founder | sop | admin | op | voice | null (rol seviyeleri: superadmin=6, founder=5, sop/admin=4, op=3, voice=2).
- Aktiflik penceresi: lastSeen >= now - 300000ms (5 dk).

6.3 Presence Silme (Leave)

```
DELETE /api/chat/rooms/{roomId}/presence
```

- lastSeen uzak geçmişe (new Date(0)) çekilir, seatIndex = -1 yapılır → user_left olayı yayılır.

SSE olayı (room_event → user_left):

```
{
  "type": "room_event",
  "event": "user_left",
  "roomId": "clr9x2abc",
  "userId": "u_7781",
  "name": "Yıldız",
  "ts": 1753267399000
}
```

7. Online Kullanıcı Sistemi

İki katman vardır: **(a)** oda bazlı online liste, **(b)** anlık sayaç.

7.1 Oda Bazlı Online Kullanıcı Listesi

```
GET /api/live/online-users?roomId={id}&roomType=voice&limit=100
Authorization: Bearer <jwt>
```

- roomType : voice → ChatPresence ; stream → VideoStreamViewer .
- limit : varsayılan 100, maks. 500.

Yanıt (voice):

```
{
  "success": true,
  "data": {
    "roomId": "clr9x2abc",
    "roomType": "voice",
    "totalCount": 37,
    "users": [
      {
        "userId": "u_7781",
        "userName": "Yıldız",
        "userImage": "https://.../avatar.jpg",
        "joinedAt": "2026-07-23T10:20:00.000Z",
        "seatIndex": 2,
        "isMicOn": false,
        "nickname": "Yıldız"
      }
    ]
  }
}
```

Yanıt (stream): aynı yapı; `seatIndex: -1`, `isMicOn: false`, `nickname: ""` döner; kaynak `Video-StreamViewer` (`leftAt=null`).

7.2 Anlık Sayaç

- Voice: SSE `presence` olayındaki `users.length` ve `POST /live/heartbeat` yanıtındaki `onlineCount`.
- Stream: SSE `viewerCount` olayı (aşağıda Bölüm 10).

7.3 Cache Katmanı

`join-room` ve `heartbeat` uçları, hızlı online durumu için Redis benzeri cache'e yazar:

`room:{roomId}:users` (set) ve `user:{userId}:presence` (hash, 10 dk TTL, `heartbeat` ile tazelenir). Bu cache yardımcı amaçlıdır; kaynak-doğru (source of truth) DB tablolarıdır.

8. Chat Sistemi

Chat mesajları `chat-events.ts` olay veri yolu üzerinden `type: 'message'` olarak yayınlanır ve SSE tarafında `type: 'messages'` (çoğul) zarfıyla client'a iletilir.

8.1 Mesaj Gönderme

Uç: `POST /api/chat/rooms/{roomId}/messages`

Body:

```
{
  "content": "Merhaba herkese 🙌",
  "type": "text",
  "replyToId": null
}
```

Sunucu mesajı oluşturur ve olay veri yoluna basar:

```
emitChatEvent(roomId, 'message', { ...message, user: { ...user, chatRole,
roleSymbol } })
```

8.2 SSE'de Mesaj Olayı (örnek JSON)

```
{
  "type": "messages",
  "messages": [
    {
      "id": "msg_01H9",
      "roomId": "clr9x2abc",
      "content": "Merhaba herkese 🙌",
      "type": "text",
      "replyToId": null,
      "createdAt": "2026-07-23T10:22:31.000Z",
      "user": {
        "id": "u_7781",
        "name": "Yıldız",
        "nickname": "Yıldız",
        "image": "https://.../avatar.jpg",
        "chatRole": "vip",
        "roleSymbol": "💎",
        "roleLevel": 3,
        "isAdmin": false
      }
    }
  ]
}
```

Not: Aynı `emitChatEvent(...'message'...)` çağrısı `POST /api/live/message` (canlı yayın alt-sohbeti) tarafından da kullanılır; payload yapısı birebir aynıdır.

8.3 Typing (Yazıyor) Göstergesi

Uç: `POST /api/chat/rooms/{roomId}/typing` — body gerekmez (kimlik oturumdan).

Sunucu `{ userId, nickname }` bilgisini 3 saniyelik pencerede tutar. SSE her poll'de aktif yazan takma adları toplayıp gönderir:

```
{
  "type": "typing",
  "users": ["Yıldız", "Mehmet"]
}
```

Boş liste (`"users": []`) hiç kimsenin yazmadığını belirtir.

8.4 Moderasyon Sistem Olayları

Uç: `POST /api/chat/rooms/{roomId}/moderation` — body `{ action, targetUserId?, reason?, duration?, text? }`.

Tüm moderasyon olayları SSE'de `type: 'system'` zarfıyla yayınlanır. Olay adları ve payload'ları:

```
{ "type": "system", "event": "USER_MUTED", "userId": "u_55", "userName": "Ali",
  "duration": 300, "moderator": "Yıldız" }
```

```
{ "type": "system", "event": "USER_UNMUTED", "userId": "u_55", "moderator": "Yıldız" }
```

```
{ "type": "system", "event": "USER_KICKED", "userId": "u_55", "userName": "Ali",  
"reason": "spam", "kickCount": 2, "moderator": "Yıldız" }
```

```
{ "type": "system", "event": "USER_BANNED", "userId": "u_55", "userName": "Ali",  
"reason": "3 kez atıldı", "moderator": "Yıldız" }
```

```
{ "type": "system", "event": "ROOM_MUTED", "moderator": "Yıldız" }
```

```
{ "type": "system", "event": "ROOM_UNMUTED", "moderator": "Yıldız" }
```

```
{ "type": "system", "event": "CHAT_CLEARED", "moderator": "Yıldız" }
```

```
{ "type": "system", "event": "ANNOUNCEMENT", "text": "Oda kuralları güncellendi",  
"ttl": 60, "moderator": "Yıldız", "timestamp": "2026-07-23T10:30:00.000Z" }
```

Otomatik ban: `USER_KICKED` olayı `kickCount` alanını taşır; 3'e ulaşıncaya sunucu otomatik olarak `USER_BANNED` yayınlar.

8.5 Hediye Olayı (chat odası)

Uç: `POST /api/chat/rooms/{roomId}/gifts` — body `{ recipientId, giftTypeId, quantity }`.
SSE'de `type: 'gift'` olarak yayınlanır:

```
{  
  "type": "gift",  
  "senderId": "u_7781",  
  "senderName": "Yıldız",  
  "recipientId": "u_55",  
  "recipientName": "Ali",  
  "giftTypeId": "gt_rose",  
  "giftName": "Gül",  
  "giftIcon": "🌹",  
  "quantity": 10,  
  "amount": 100,  
  "currencyType": "jeton"  
}
```

(Ayrıntılı hediye akışı için bkz. Bölüm 16.)

9. Voice Room (Sesli Oda) Akışı

Sesli oda, chat odasının (`ChatRoom` , `roomType: 'voice'`) üzerine kurulu 15 koltuklu (0-14) sesli yayın katmanıdır. Ses medyası **Tencent RTC (TRTC)** üzerinden taşınır; koltuk/mikrofon/sahiplik durumu ise

`voice-room-events.ts` aracılığıyla `chat-events` veri yoluna `type: 'room'` olarak basılır ve SSE'de `type: 'room_event'` zarfıyla iletilir.

9.1 Odaya Katılma (özet)

Tam akış Bölüm 12'de. Kısaca: `POST /api/live/join-room { roomId, roomType: "voice" }` → TRTC kimlik bilgileri + oda durumu + koltuk haritası döner. Ardından SSE stream'e (`/api/chat/rooms/{roomId}/stream`) bağlanılır.

9.2 Oda Olayı Türleri (RoomEventKind)

`voice-room-events.ts` şu olay türlerini üretir; hepsi SSE'de `type: "room_event"` altında `event` alanıyla ayrışır:

event	Ne zaman	Tetikleyen uç
<code>user_joined</code>	Kullanıcı odaya girdi	<code>POST /chat/rooms/{id}/presence</code>
<code>user_left</code>	Kullanıcı odadan çıktı	<code>DELETE /chat/rooms/{id}/presence</code>
<code>mic_changed</code>	Mikrofon açıldı/kapandı (koltuğa çıkış/iniş)	<code>POST /chat/rooms/{id}/voice</code>
<code>seat_changed</code>	Koltuk atandı/değiştirdi	<code>PATCH /chat/rooms/{id}/seats</code>
<code>owner_changed</code>	Oda sahipliği devredildi	<code>POST /chat/rooms/{id}/transfer-ownership</code>
<code>room_closed</code>	Oda kapatıldı	Oda kapatma ucu

9.3 Örnek Payload'lar

`user_joined`

```
{
  "type": "room_event",
  "event": "user_joined",
  "roomId": "clr9x2abc",
  "userId": "u_7781",
  "name": "Yıldız",
  "image": "https://.../avatar.jpg",
  "seatIndex": -1,
  "ts": 1753265000000
}
```

`user_left`

```
{
  "type": "room_event",
  "event": "user_left",
  "roomId": "clr9x2abc",
  "userId": "u_7781",
  "name": "Yıldız",
  "ts": 1753265050000
}
```

mic_changed (koltuğa çıkış: micOn: true , koltuktan iniş: micOn: false)

```
{
  "type": "room_event",
  "event": "mic_changed",
  "roomId": "clr9x2abc",
  "userId": "u_7781",
  "name": "Yıldız",
  "image": "https://.../avatar.jpg",
  "micOn": true,
  "seatIndex": 2,
  "ts": 1753265100000
}
```

seat_changed

```
{
  "type": "room_event",
  "event": "seat_changed",
  "roomId": "clr9x2abc",
  "userId": "u_55",
  "name": "Ali",
  "seatIndex": 5,
  "previousSeatIndex": 2,
  "ts": 1753265150000
}
```

owner_changed

```
{
  "type": "room_event",
  "event": "owner_changed",
  "roomId": "clr9x2abc",
  "newOwnerId": "u_55",
  "newOwnerName": "Ali",
  "ts": 1753265200000
}
```

room_closed

```
{
  "type": "room_event",
  "event": "room_closed",
  "roomId": "clr9x2abc",
  "ts": 1753265300000
}
```

9.4 Mikrofon (Koltuk) Kontrolü

Uç: POST /api/chat/rooms/{roomId}/voice

Body:

```
{ "type": "join" }
```

("leave" mikrofonu kapatıp koltuktan iner.)

Sunucu VoiceSession kaydını upsert eder (agoraUid alanı geriye dönük uyumluluk için numericUid değeriyle doldurulur) ve emitMicChanged(...) çağırır. Yetki: yalnızca oda sahibi, global admin veya voice+ rolüne sahip kullanıcılar koltuğa çıkabilir.

Yanıt:

```
{
  "success": true,
  "data": {
    "seatIndex": 2,
    "micOn": true,
    "numericUid": 483920114
  }
}
```

9.5 Koltuk Haritası

Uç: GET /api/chat/rooms/{roomId}/seats — 15 koltukluk (0-14) harita döner:

```
{
  "success": true,
  "data": {
    "seats": [
      { "seatIndex": 0, "userId": "u_7781", "userName": "Yıldız", "image": "https://.../a.jpg", "micOn": true, "isThrone": true },
      { "seatIndex": 1, "userId": null },
      { "seatIndex": 2, "userId": "u_55", "userName": "Ali", "image": "https://.../b.jpg", "micOn": false, "isThrone": false }
    ]
  }
}
```

Koltuk atama: PATCH /api/chat/rooms/{roomId}/seats

```
{ "targetUserId": "u_55", "seatIndex": 5, "forceThrone": false, "forceAssign": false }
```

9.6 Sahiplik Devri

Uç: POST /api/chat/rooms/{roomId}/transfer-ownership — body { newOwnerId }. Başarıda owner_changed olayı yayınlanır (bkz. 9.3).

10. Live Stream (Canlı Yayın) Akışı

Video canlı yayınları `VideoStream` tablosu + `stream-events.ts` veri yolu ile çalışır. SSE ucu:

`GET /api/video-streams/{streamId}/stream` (poll 1 sn). Video/ses medyası TRTC üzerinden taşınır; SSE yalnızca meta olayları (mesaj, izleyici sayısı, hediye, PK, yayın sonu) taşır.

Önemli: Bu SSE ucu olayları **ham** (raw) iletir — her olay kendi `type` alanını içinde taşır. Client `type` alanına göre ayrıştırır.

10.1 Bağlantı Olayı

Bağlanınca sırayla `connected` ve ilk `viewerCount` gönderilir:

```
{ "type": "connected", "streamId": "vs_9021" }
```

10.2 İzleyici Sayısı (viewerCount)

Katılma/ayrılmada güncellenir:

```
{
  "type": "viewerCount",
  "streamId": "vs_9021",
  "viewerCount": 128,
  "viewers": [
    { "userId": "u_7781", "name": "Yıldız", "image": "https://.../a.jpg" },
    { "userId": "u_55", "name": "Ali", "image": "https://.../b.jpg" }
  ]
}
```

10.3 Yayın Mesajı / Yorum (streamMessage)

Uçlar: `POST /api/video-streams/{streamId}/messages` ve `POST /api/video-streams/{streamId}/comments`.

```
{
  "type": "streamMessage",
  "streamId": "vs_9021",
  "message": {
    "id": "sm_2210",
    "streamId": "vs_9021",
    "content": "Harika yayın!",
    "createdAt": "2026-07-23T10:40:00.000Z",
    "user": {
      "id": "u_55",
      "name": "Ali",
      "nickname": "Ali",
      "image": "https://.../b.jpg",
      "role": "user",
      "membership": "premium"
    }
  }
}
```

10.4 Yayın Sonu (streamEnded)

Uçlar: `PATCH /api/video-streams/{streamId}` (status=ended) veya `POST /api/video-streams/{streamId}/end`.

```
{
  "type": "streamEnded",
  "event": "STREAM_ENDED",
  "streamId": "vs_9021"
}
```

10.5 Hediye (stream)

POST /api/live/gift/send veya POST /api/video-streams/{streamId}/gifts → bkz. Bölüm 16. Örnek:

```
{
  "type": "gift",
  "streamId": "vs_9021",
  "gift": {
    "id": "g_5521",
    "senderName": "Yıldız",
    "senderImage": "https://.../a.jpg",
    "giftName": "Kalp",
    "giftIcon": "❤️",
    "quantity": 5,
    "totalPrice": 250
  }
}
```

11. Tencent RTC (TRTC) Token Üretimi

Sesli oda ve canlı yayın medyası TRTC üzerinden taşınır. Client'ın TRTC SDK'sına bağlanabilmesi için sunucu tarafında UserSig imzası üretilir. Gizli anahtar (SecretKey) **asla** client'a gönderilmez.

11.1 Ortam Değişkenleri

Sunucu, aşağıdaki fallback zinciriyle değerleri okur:

Amaç	Değişkenler (öncelik sırası)
SDK App ID	TRTC_SDK_APP_ID → TENCENT_TRTC_SDK_APP_ID
Secret Key	TRTC_SDK_SECRET_KEY → TRTC_SECRET_KEY → TENCENT_TRTC_SECRET_KEY
Sig geçerlilik (sn)	TRTC_EXPIRE (varsayılan 86400 = 24 saat)

İmza tls-sig-api-v2 paketiyle üretilir: new Api(sdkAppId, secretKey).genSig(userId, expireTime) .

11.2 Deterministik Kimlik Yardımcıları (lib/trtc-room.ts)

Hem web hem Flutter **aynı** deterministik fonksiyonları kullanmalıdır (parite şartı):

- voiceTrtcRoomId(chatRoomId) → "voice_room_" + chatRoomId (zaten önekliyse tekrar eklemes — idempotent).
- userIdToNumericUid(userId) → deterministik hash % 1000000000 (TRTC sayısal UID gerektirir).

11.3 POST /api/trtc/usersig

Misafir izleyicilere de (ör. viewer_xxx) izin verir.

Body:

```
{ "userId": "u_7781", "roomId": "clr9x2abc" }
```

Yanıt:

```
{
  "sdkAppId": 1400000001,
  "userId": "u_7781",
  "userSig": "eJwtzMEK...imzali_string...",
  "roomId": "clr9x2abc",
  "trtcRoomId": "voice_room_clr9x2abc",
  "numericUid": 483920114
}
```

11.4 POST /api/trtc/token

JWT zorunludur (mobil Bearer veya web oturumu).

Body:

```
{ "roomId": "clr9x2abc", "role": "anchor" }
```

Başarılı Yanıt:

```
{
  "success": true,
  "data": {
    "sdkAppId": 1400000001,
    "userId": "u_7781",
    "userSig": "eJwtzMEK...imzali_string...",
    "roomId": "clr9x2abc",
    "trtcRoomId": "voice_room_clr9x2abc",
    "numericUid": 483920114,
    "expireTime": 86400,
    "role": "anchor"
  }
}
```

Hata Zarfı:

```
{
  "success": false,
  "error": { "code": "UNAUTHORIZED", "message": "Oturum bulunamadı" }
}
```

11.5 Client Bağlanma Sırası

1. join-room (Bölüm 12) veya trtc/token çağrılır → sdkAppId , userSig , trtcRoomId , numericUid alınır.
2. TRTC SDK enterRoom({ sdkAppId, userId, userSig, roomId: trtcRoomId, ... }) ile bağlanır.

3. Yayıncı/koltuktaki kullanıcı `startLocalAudio` / `startLocalVideo` ; izleyici yalnızca uzak akışları oynatır.

12. Join / Leave Akışları

12.1 Odaya Katılma — `POST /api/live/join-room`

Bileşik (compound) uçtur: hem TRTC kimlik bilgilerini üretir hem de oda anlık görüntüsünü döner.

Body:

```
{ "roomId": "clr9x2abc", "roomType": "voice", "nickname": "Yıldız" }
```

(`roomType` : "stream" veya "voice" ; `nickname` opsiyonel.)

Başarılı Yanıt (kısaltılmış):

```
{
  "success": true,
  "data": {
    "room": {
      "id": "clr9x2abc",
      "roomId": "1001",
      "slug": "yildiz-odasi",
      "name": "Yıldız Odası",
      "nameEn": "Star Room",
      "title": "Akşam Sohbeti",
      "description": "Herkes'e açık sohbet",
      "status": "live",
      "category": "sohbet",
      "thumbnailUrl": "https://.../t.jpg",
      "backgroundUrl": "https://.../bg.jpg",
      "isImageMode": false,
      "isMuted": false,
      "roomType": "voice",
      "roomAccessType": "public",
      "welcomeMessage": "Hoş geldiniz!",
      "pinnedAnnouncement": null,
      "viewerCount": 37,
      "likeCount": 210,
      "startedAt": "2026-07-23T09:00:00.000Z",
      "host": { "id": "u_7781", "name": "Yıldız", "image": "https://.../a.jpg" }
    },
    "trtc": {
      "sdkAppId": 14000000001,
      "userId": "u_7781",
      "userSig": "eJwtzMEK...",
      "roomId": "voice_room_clr9x2abc",
      "expireTime": 86400
    },
    "user": {
      "id": "u_7781", "name": "Yıldız", "image": "https://.../a.jpg",
      "role": "user", "isHost": true, "isBanned": false
    },
    "participants": [ { "userId": "u_55", "name": "Ali", "image": "https://.../b.jpg", "seatIndex": 2, "micOn": false } ],
    "seats": [ { "seatIndex": 0, "userId": "u_7781" } ],
    "giftRanking": [ { "userId": "u_55", "name": "Ali", "totalAmount": 1500 } ]
  }
}
```

Hata kodları: ROOM_NOT_FOUND (404), STREAM_ENDED (410), BANNED (403).

Sunucu ayrıca izleyici/presence kaydını upsert eder ve `user_joined` (voice) / `viewerCount` (stream) olayını yayınlar.

12.2 Odadan Ayrılma — POST /api/live/leave-room

Body:

```
{ "roomId": "clr9x2abc", "roomType": "voice" }
```

Davranış:

- **Stream + host ayrılırsa:** yayın sonlandırılır (`status: ended`), tüm izleyiciler “ayrıldı” işaretlenir, `streamEnded` yayınlanır.
- **Stream + izleyici ayrılırsa:** izleyici “ayrıldı” işaretlenir, `viewerCount` güncellenir.

- **Voice:** presence kaydında `lastSeen = new Date(0)`, `seatIndex = -1` yapılır ve `user_left` yayınlanır.

Yanıt:

```
{ "success": true, "data": { "left": true } }
```

12.3 Oda Durumu Anlık Görüntüsü — GET /api/chat/rooms/{roomId}/state

SSE'ye bağlanmadan önce/sonra tam durum çekmek için:

```
{
  "success": true,
  "data": {
    "room": { "id": "clr9x2abc", "name": "Yıldız Odası", "status": "live",
    "roomType": "voice" },
    "participants": [ { "userId": "u_55", "name": "Ali", "seatIndex": 2, "micOn": false } ],
    "seats": [ { "seatIndex": 0, "userId": "u_7781" } ],
    "onlineCount": 37,
    "me": { "id": "u_7781", "isHost": true, "chatRole": "owner", "micOn": true, "seatIndex": 0 },
    "trtc": { "sdkAppId": 1400000001, "trtcRoomId": "voice_room_clr9x2abc", "numericUid": 483920114 }
  }
}
```

12.4 Presence (Katılımcı Kaydı)

- **Katıl:** POST /api/chat/rooms/{roomId}/presence — body { "nickname": "Yıldız", "seatIndex": -1 } → user_joined .
- **Ayrıl:** DELETE /api/chat/rooms/{roomId}/presence → user_left .

12.5 Önerilen Client Akışı

1. join-room → TRTC kimlik + oda anlık görüntüsü.
2. TRTC enterRoom(...) .
3. SSE stream'e bağlan (/chat/rooms/{id}/stream veya /video-streams/{id}/stream).
4. Her 10 sn POST /live/heartbeat (liveness).
5. Çıkışta: SSE kapat → TRTC exitRoom() → POST /live/leave-room .

13. PK Battle (Rekabet) Sistemi

İki chat/voice odasının puan yarışıdır. Gerçek-zamanlı akış `chat-events.ts` veri yolundaki `type: 'pk'` olayları üzerinden yürür (her iki odaya da yayınlanır) ve SSE'de `type: 'pk'` zarfıyla iletilir.

Not: video-streams/pk uçları herhangi bir SSE olayı **yaymaz**; video yayınlarındaki PK durumu da chat-events pk kanalı üzerinden aktarılır.

Uç: POST /api/chat/rooms/{roomId}/pk — { action, ... } . action değerleri: create , accept , reject , cancel , end . Ayrıca POST /api/chat/rooms/{roomId}/pk/score puan günceller.

13.1 created (davet)

action: create → her iki odaya yayınlanır. PK_TIMEOUT = 60 sn .

```
{
  "type": "pk",
  "battleId": "pk_3301",
  "action": "created",
  "room1Id": "clr9x2abc",
  "room2Id": "clr8y1def",
  "user1Id": "u_7781",
  "user2Id": "u_9002",
  "challengerName": "Yıldız",
  "duration": 300,
  "status": "pending",
  "expiresAt": "2026-07-23T10:51:00.000Z",
  "timeoutSeconds": 60
}
```

13.2 started (kabul edildi)

```
{
  "type": "pk",
  "battleId": "pk_3301",
  "action": "started",
  "room1Id": "clr9x2abc",
  "room2Id": "clr8y1def",
  "user1Id": "u_7781",
  "user2Id": "u_9002",
  "score1": 0,
  "score2": 0,
  "duration": 300,
  "status": "active",
  "startedAt": "2026-07-23T10:50:20.000Z",
  "endTime": "2026-07-23T10:55:20.000Z"
}
```

13.3 score_update (puan)

POST /api/chat/rooms/{roomId}/pk/score ve hediye ile otomatik puanlama tetikler:

```
{
  "type": "pk",
  "battleId": "pk_3301",
  "action": "score_update",
  "score1": 1200,
  "score2": 800,
  "room1Id": "clr9x2abc",
  "room2Id": "clr8y1def",
  "addedAmount": 250,
  "addedSide": "room1"
}
```

Hediye REST yanıtı ayrıca pkUpdate: { battleId, score1, score2 } alanını döner (bkz. Bölüm 16).

13.4 rejected / cancelled

```
{
  "type": "pk",
  "battleId": "pk_3301",
  "action": "rejected",
  "room1Id": "clr9x2abc",
  "room2Id": "clr8y1def",
  "user1Id": "u_7781",
  "user2Id": "u_9002",
  "status": "rejected"
}
```

(action: "cancelled" / status: "cancelled" iptal için aynı yapıda.)

13.5 completed (bitiş)

action: end veya süre dolunca:

```
{
  "type": "pk",
  "battleId": "pk_3301",
  "action": "completed",
  "room1Id": "clr9x2abc",
  "room2Id": "clr8y1def",
  "user1Id": "u_7781",
  "user2Id": "u_9002",
  "score1": 1200,
  "score2": 800,
  "winnerId": "u_7781",
  "status": "completed"
}
```

14. Co-Host (Ortak Yayın) Sistemi

Video canlı yayınına misafir yayıncı (co-broadcaster) davet/kabul sistemidir. `streamCoBroadcaster` tablosu ile yönetilir.

ÖNEMLİ (parite notu): Co-Host durumu **SSE ile yayınlanmaz**. Durum senkronizasyonu **GET polling + push bildirimleri** üzerinden yürür; ses/görüntü medyası ise TRTC üzerinden taşınır. Flutter tarafı da aynı polling modelini uygulamalıdır.

Uç: `POST /api/video-streams/{streamId}/co-broadcast — { action, targetUserId? } .`

14.1 Aksiyonlar

action	Kim çağırır	Etki
request	İzleyici	Yayıncıya katılma isteği (host'a bildirim)
invite	Host	Bir izleyiciyi davet eder (MAX_GUESTS = 8)
approve	Host	İsteği/daveti onaylar → active
reject_request	Host	İsteği reddeder
mute / unmute	Host	Misafirin mikrofonunu kapatır/açar
video_off / video_on	Host	Misafirin kamerasını kapatır/açar
remove	Host	Misafiri yayından çıkarır → ended

Durum akışı: requested / invited → active → ended .

14.2 Davet İsteği (örnek)

İstek (izleyici):

```
{ "action": "request" }
```

Davet (host):

```
{ "action": "invite", "targetUserId": "u_55" }
```

14.3 Kabul/Ret — PATCH /api/video-streams/{streamId}/co-broadcast

Davet edilen kullanıcı yanıtlar:

```
{ "action": "accept" }
```

("reject" reddeder.)

14.4 Durum Sorgulama — GET /api/video-streams/{streamId}/co-broadcast

Polling ile güncel co-broadcaster listesi çekilir:

```
{
  "success": true,
  "data": {
    "coBroadcasters": [
      {
        "id": "cb_7781",
        "streamId": "vs_9021",
        "userId": "u_55",
        "userName": "Ali",
        "userImage": "https://.../b.jpg",
        "status": "active",
        "isMuted": false,
        "isVideoOff": false,
        "numericUid": 771029341,
        "joinedAt": "2026-07-23T10:58:00.000Z"
      }
    ],
    "maxGuests": 8
  }
}
```

Client bu listeyi periyodik (ör. 2-3 sn) çekerek co-host panelini günceller; onaylanan misafir TRTC’de yayıncı rolüne geçer.

15. Music Queue (DJ / Müzik Kuyruğu) Sistemi

Sesli/chat odalarında ortak müzik dinleme özelliğidir. Mimari **YouTube embed** tabanlıdır (ham ses akışı çıkarımı yapılmaz). Durum `chat-dj-events.ts` içindeki `djEventStore` (oda başına tek “en güncel” olay) ile tutulur ve SSE’de `type: 'dj'` olarak iletilir.

15.1 Kuyruk Depolama Modeli

Şarkı istekleri ayrı bir tablo yerine `ChatMessage` kayıtları olarak saklanır:

- Ücretli istek öneki: `[SONG_REQUEST_PAID]` , ücretsiz: `[SONG_REQUEST_FREE]` .
- Alanlar pipe (`|`) ile ayrılır. Çalınmış öğeler `[PLAYED]` ile işaretlenir ve kuyruktan filtrelendir.
- Ücretli istekler kuyruğun başına sıralanır.

15.2 Şarkı İsteği — `POST /api/chat/rooms/{roomId}/song-request`

Body:

```
{
  "videoId": "dQw4w9WgXcQ",
  "title": "Örnek Şarkı",
  "dedication": "Herkes",
  "note": "Favorim 🎵",
  "duration": 213,
  "priority": false,
  "requestType": "video"
}
```

İstek işlenince sunucu `emitDjUpdate(roomId)` çağırır ve güncel DJ payload’ı yayınlanır.

15.3 DJ Payload (SSE type: 'dj')

SSE stream'e ilk bağlanışta tam payload `buildDjPayload(roomId)` ile gönderilir; sonraki poll'lerde yalnızca `getLatestDjEvent(roomId, ts)` ile değişiklik gönderilir.

```
{
  "type": "dj",
  "event": "QUEUE_UPDATED",
  "playing": true,
  "nowPlaying": {
    "videoId": "dQw4w9WgXcQ",
    "title": "Örnek Şarkı",
    "startedAt": "2026-07-23T11:00:00.000Z",
    "startedAtMs": 1753268400000,
    "elapsedSeconds": 42,
    "duration": 213,
    "embedUrl": "https://www.youtube.com/embed/dQw4w9WgXcQ?autoplay=1&start=42&enablejsapi=1&playsinline=1"
  },
  "musicUrl": "https://www.youtube.com/watch?v=dQw4w9WgXcQ",
  "embedUrl": "https://www.youtube.com/embed/dQw4w9WgXcQ?autoplay=1&start=42&enablejsapi=1&playsinline=1",
  "musicQueue": [
    {
      "id": "msg_9001",
      "videoId": "abcd1234",
      "title": "Sıradaki Şarkı",
      "dedication": "Ali'ye",
      "note": "",
      "duration": 180,
      "requestType": "video",
      "isPaid": true,
      "userId": "u_55",
      "userName": "Ali",
      "createdAt": "2026-07-23T11:01:00.000Z"
    }
  ],
  "queueLength": 1
}
```

15.4 Senkronizasyon Mantığı

`elapsedSeconds`, `startedAtMs` üzerinden sunucu zamanına göre hesaplanır; tüm client'lar `embedUrl` içindeki `start=` parametresiyle aynı konumdan oynatarak senkron kalır. `emitDjUpdate` çağıran uçlar: song-request, music (oynat/duraklat/geç), sıradan çıkarma.

16. Gift Event (Hediye) Sistemi

Hediyeler jeton/coin ile gönderilir; alıcının kazancını ve (aktifse) PK puanını günceller. Hedefe göre farklı veri yollarına yayınlanır.

16.1 Chat / Voice Odasına Hediye

Uç: POST /api/chat/rooms/{roomId}/gifts — body { recipientId, giftTypeId, quantity } .
SSE'de type: 'gift' :

```
{
  "type": "gift",
  "senderId": "u_7781",
  "senderName": "Yıldız",
  "recipientId": "u_55",
  "recipientName": "Ali",
  "giftTypeId": "gt_rose",
  "giftName": "Gül",
  "giftIcon": "🌹",
  "quantity": 10,
  "amount": 100,
  "currencyType": "jeton"
}
```

16.2 Canlı Yayına Hediye — POST /api/live/gift/send

Hedef stream ise `stream-events` , voice oda ise `chat-events` veri yoluna yayınlanır.

Stream’e (SSE type: 'gift'):

```
{
  "type": "gift",
  "streamId": "vs_9021",
  "gift": {
    "id": "g_5521",
    "senderName": "Yıldız",
    "senderImage": "https://.../a.jpg",
    "giftName": "Kalp",
    "giftIcon": "❤️",
    "quantity": 5,
    "totalPrice": 250
  }
}
```

Voice odaya (SSE type: 'gift'):

```
{
  "type": "gift",
  "roomId": "clr9x2abc",
  "gift": {
    "id": "g_5522",
    "senderName": "Yıldız",
    "senderImage": "https://.../a.jpg",
    "recipientName": "Ali",
    "giftName": "Kalp",
    "giftIcon": "❤️",
    "quantity": 5,
    "totalPrice": 250
  }
}
```

16.3 Video Stream Hediyesi — POST /api/video-streams/{streamId}/gifts

```
{
  "type": "gift",
  "streamId": "vs_9021",
  "gift": {
    "id": "g_5523",
    "senderName": "Yıldız",
    "giftName": "Taç",
    "giftIcon": "👑",
    "quantity": 1,
    "totalPrice": 1000
  }
}
```

Gönderen izleyici listesinden hariçse (`senderExcluded`) `totalPrice` 0 gönderilir.

16.4 Hediye ile Otomatik PK Puanı

Aktif bir PK sırasında gönderilen hediye, gönderenin odasının puanını artırır. REST yanıtı:

```
{
  "success": true,
  "data": {
    "giftSent": true,
    "pkUpdate": { "battleId": "pk_3301", "score1": 1450, "score2": 800 }
  }
}
```

Ayrıca `type: 'pk' / action: 'score_update'` olayı yayınlanır (bkz. 13.3).

17. Canlı Fal Seansı (Session Room) Akışı

Kullanıcı ile falcı arasındaki birebir canlı fal seansıdır. `room-events.ts` iki ayrı olay akışı yönetir: **seans olayları** (session başına) ve **falcı olayları** (teller başına, gelen istekler için).

17.1 Seans SSE — GET /api/room/{sessionId}/stream

Poll 1 sn. Yalnızca seansın kullanıcısı veya falcısı bağlanabilir.

ÖNEMLİ: Bu uç `event.data` içeriğini **ham** iletir — ayrı bir `type` alanı **yoktur**. Client, gelen alanlara bakarak olay türünü çıkarır (ör. `timerStartedAt` varsa timer başlamıştır; `actualMinutesUsed` varsa seans bitmiştir).

Bağlantı olayı (zengin):

```
{
  "sessionId": "sess_4410",
  "isUser": true,
  "isTeller": false,
  "status": "active",
  "timerStarted": true,
  "timerStartedAt": "2026-07-23T11:10:00.000Z",
  "maxMinutes": 15,
  "minutesUsed": 3
}
```

17.2 Seans Mesajı — POST /api/room/{sessionId}/messages

```
{
  "id": "rm_2201",
  "senderId": "u_7781",
  "message": "Merhaba, fincanınızı görebilir miyim?",
  "createdAt": "2026-07-23T11:11:00.000Z"
}
```

17.3 Zamanlayıcı Başladı (timer_started)

```
{ "timerStartedAt": "2026-07-23T11:10:00.000Z" }
```

17.4 Süre Uzatıldı (time_extended)

```
{ "addedMinutes": 5, "newMaxMinutes": 20, "by": "teller" }
```

17.5 Seans Bitti (session_ended)

```
{
  "actualMinutesUsed": 12,
  "actualCost": 240,
  "refundAmount": 60,
  "tellerEarnings": 168,
  "commissionAmount": 72,
  "tellerEarningsTL": 84.0,
  "clientSpentTL": 120.0,
  "jetonTLRate": 0.5,
  "endedBy": "teller"
}
```

Seans bitiminden 10 sn sonra sunucu `clearRoomEvents(sessionId)` ile tamponu temizler.

17.6 Falcı İstek Akışı — GET /api/fortune-tellers/sessions/stream

Poll 3 sn. Yalnızca onaylı ve banlı olmayan falcı bağlanabilir. Bağlanınca `connected` + `pending_sessions` (bağlanışta ve her 15 sn) gönderilir.

connected:

```
{ "type": "connected", "tellerId": "t_9002", "isOnline": true }
```

pending_sessions:

```
{
  "type": "pending_sessions",
  "sessions": [
    {
      "sessionId": "sess_4410",
      "userId": "u_7781",
      "userName": "Yıldız",
      "fortuneType": "kahve",
      "duration": 15,
      "creditsCharged": 300,
      "createdAt": "2026-07-23T11:09:00.000Z"
    }
  ]
}
```

17.7 Yeni Seans İsteği (session_request)

POST /api/fortune-tellers/session → falcıya:

```
{
  "sessionId": "sess_4411",
  "userId": "u_55",
  "userName": "Ali",
  "fortuneType": "tarot",
  "duration": 10,
  "creditsCharged": 200,
  "createdAt": "2026-07-23T11:15:00.000Z"
}
```

17.8 Seans İptal/Ret (session_cancelled)

PATCH /api/fortune-tellers/sessions/{sessionId} (action cancel / reject):

```
{ "sessionId": "sess_4411", "action": "cancel" }
```

18. WebRTC Signaling (İşaretleşme) Sistemi

TRTC'nin yanı sıra, birebir/özel bağlantılarda kullanılan WebRTC işaretleşme (offer/answer/ICE) kanalı. İki ayrı uç mevcuttur: fal seansları için `room/signal`, video yayınları için `video-streams/signal`. Sinyaller DB'de saklanır ve alıcı tarafından poll ile çekilir (SSE kullanılmaz).

18.1 Seans Sinyali — POST /api/room/signal

DB tablosu: `RoomSignal`. `signalData` DB'ye JSON string olarak yazılır.

Body:

```
{
  "sessionId": "sess_4410",
  "receiverId": "u_55",
  "signalType": "offer",
  "signalData": { "sdp": "v=0\r\no=- ...", "type": "offer" }
}
```

(signalType : offer | answer | ice-candidate .)

Alma — GET /api/room/signal?sessionId=sess_4410

Kullanıcı için işlenmemiş (pending) sinyalleri döner ve işlenmiş olarak işaretler:

```
{
  "success": true,
  "signals": [
    {
      "id": "sig_101",
      "senderId": "u_7781",
      "signalType": "offer",
      "signalData": { "sdp": "v=0\r\no=- ...", "type": "offer" },
      "createdAt": "2026-07-23T11:12:00.000Z"
    }
  ]
}
```

Temizleme — DELETE /api/room/signal?sessionId=sess_4410

Kullanıcının sinyallerini temizler (yeniden bağlanma / reconnect senaryosu için).

18.2 Yayın Sinyali — video-streams/signal

Alma — GET /api/video-streams/signal?streamId=vs_9021&recipientId=u_55

Son 60 sn içindeki, en çok 50 sinyali döner ve işlenmiş işaretler:

```
{
  "success": true,
  "signals": [
    {
      "id": "sig_552",
      "type": "ice-candidate",
      "senderId": "u_7781",
      "data": { "candidate": "candidate:842...", "sdpMid": "0", "sdpMLineIndex": 0 },
      "createdAt": "2026-07-23T11:20:00.000Z"
    }
  ]
}
```

Gönderme — POST /api/video-streams/signal

```
{
  "streamId": "vs_9021",
  "type": "offer",
  "receiverId": "u_55",
  "data": { "sdp": "v=0\r\no=- ...", "type": "offer" }
}
```

Misafir (guest) gönderene de izin verilir.

Temizleme — DELETE /api/video-streams/signal

60 sn'den eski sinyalleri temizler.

18.3 Not — TRTC vs WebRTC Signaling

Uygulamanın ana sesli/görüntülü medyası **TRTC** üzerinden taşınır (Bölüm 11). Bu WebRTC signaling katmanı, TRTC dışı özel P2P senaryoları ve geriye dönük uyumluluk için korunur. Flutter tarafında TRTC SDK kullanıldığında bu signaling uçları çoğunlukla gerekmez; yalnızca P2P özel bağlantı gerektiren akışlarda devreye girer.

19. Flutter ↔ Web Parite Kontrol Listesi

Aşağıdaki noktalar iki platformun **birebir aynı** çalışması için kritik:

1. **TRTC oda kimliği:** `voiceTrtcRoomId(chatRoomId) = "voice_room_" + id` . İki platform da aynı öneki kullanmalı.
2. **Sayısal UID:** `userIdToNumericUid(userId)` deterministik hash `% 1000000000` . Aynı algoritma zorunlu.
3. **SSE poll aralıkları:** chat 2 sn, video-stream 1 sn, room 1 sn, teller-sessions 3 sn, notifications 5 sn.
4. **Heartbeat:** 10 sn'de bir `POST /live/heartbeat` ; stale eşiği 60 sn; presence penceresi 5 dk.
5. **SSE keep-alive:** sunucu 15 sn'de bir `: heartbeat\n\n` yorumu gönderir; client bunu yok saymalı.
6. **Reconnect:** `Last-Event-ID` / `since` zaman damgası ile kaldığı yerden devam; tampon TTL'leri (chat 2 dk, room 5 dk) aşılsa tam durum `state` ucundan yeniden çekilir.
7. **Olay zarfı farkları:** chat/video-stream olayları `type` alanı taşır; **room (fal seansı)** SSE'si ham veri gönderir (type yok) — alanlara göre ayrıştır.
8. **PK:** hem chat hem video PK akışı chat-events `type: 'pk'` üzerinden gelir; `video-streams/pk` SSE yaymaz.
9. **Co-Host:** SSE yok — GET polling + push bildirimi.
10. **DJ/Müzik:** YouTube embed + `elapsedSeconds / start=` ile sunucu-saat senkronizasyonu.
11. **Auth:** tüm uçlar çift kimlik doğrulama — mobil `Authorization: Bearer <JWT>` veya web oturumu.

Bu doküman `nextjs_space` kaynak kodundaki gerçek uçlar ve olay veri yolları (event bus) temel alınarak hazırlanmıştır. Kodda karşılığı olmayan hiçbir olay/uç eklenmemiştir.